

Outliers, Inconsistencies, and Data Quality

Finding the values that do not belong — and deciding what to do about them

Data Wrangling

Fall 2026

Opening (2 min).

Hook the room before any definitions. Ask out loud: *“If one runner in a 15-person race finished in 47 minutes and everyone else took about 100 minutes — did she win, or did she cut the course?”*

Let two or three students answer. You will get both responses, and that disagreement *is* the lesson: the number alone cannot tell you. Statistics finds the value; **domain knowledge** decides its fate.

Say plainly what today is not: it is not a set of rules for deleting inconvenient data. Deleting an outlier you do not understand is how people accidentally erase the most important observation in the dataset.

Housekeeping: notebook is outliers_data_wrangling.ipynb; tell them to open it now and run the first cell so imports are warm.

By the end of today you can:

- 1. **Define** an outlier and name its three types
- 2. **Detect** outliers three ways: z-score, IQR, modified z-score
- 3. **Explain** why the z-score fails on small samples
- 4. **Decide** whether to drop, cap, keep, or fix a value
- 5. **Repair** inconsistencies (units, duplicates, categories)
- 6. **Score** a dataset on six quality dimensions

Three stories we will solve:

- 1. **The Bison Half-Marathon**
A 47-minute half marathon
- 2. **The \$1 Apartment**
A typo that hides in plain sight
- 3. **The Ice Cream Truck**
An outlier no single column can see

Rule for today
Every flagged value gets *investigated*, not deleted.

Roadmap (1 min). Read the six objectives quickly — do not dwell.

Spend your energy on the right column. Naming the three examples up front makes them memorable, and students will start guessing answers before you get there, which is exactly what you want.

Emphasize the box: *investigated, not deleted*. You will return to this sentence at least four times today. It is the single idea you want them to keep if they forget everything else.

Timing plan for the session: definitions & detection $\approx$ 20 min, three examples $\approx$ 30 min, handling $\approx$ 10 min, inconsistencies & quality $\approx$ 15 min, practice $\approx$ 15 min.

What Is an Outlier?

Fifteen half-marathon finishing times (minutes). Fourteen runners finished between 92 and 121. One finished in **47**.

	With 47	Without 47	Change
Mean	101.8	105.7	−3.9 min
Median	104.0	105.0	−1.0 min
Standard deviation	17.37	8.79	+98%

The damage is not where you think

The mean moved by 4 minutes. The *standard deviation nearly doubled*. Every confidence interval, every *t*-test, every “±2 SD” rule built on this dataset is now wrong — and one single row did it.

Motivation (3 min). Do not rush this table. It is the argument for the whole lecture.

Walk the three rows in order and ask the class to predict each change *before* you reveal it:

- Mean: they will correctly guess it drops.
- Median: ask “how much does the median move?” Most will overestimate. It moves 1 minute. The median barely notices — this is your first hint that some statistics are *robust* and some are not.
- SD: this is the surprise. Nearly doubled. Let that land.

Key line to say out loud: “Most people worry that outliers move the average. The bigger problem is that they inflate the spread, and spread is what every significance test divides by.”

If someone asks why the SD doubles: the SD squares deviations, so a point that is 55 units away contributes $55^2 = 3025$ while a typical point contributes maybe 64. One point supplies most of the variance.

Definition

Outlier

An **outlier** is an observation that lies at an abnormal distance from the other values in a dataset, given some definition of “distance” and “abnormal.”

Notice what this definition does *not* say:

- It does not say the value is **wrong**.
- It does not say the value should be **removed**.
- It does not give a single **threshold** — “abnormal” is a choice you make and must defend.

Two famous outliers

Kept: In 1985 the Antarctic ozone hole readings were so low that NASA’s software had been discarding them automatically as errors for years. They were real.

Removed: A hospital records a patient age of 999. That is a sentinel value for “unknown,” not a very old patient.

Definition (3 min).

Put weight on the three “does not say” bullets. Students arrive believing outlier = bad data. That belief is what you are here to break.

The ozone story is worth telling properly, because it is the best argument you have. British Antarctic Survey scientists found the hole with ground instruments. NASA had satellite data showing it too, but low readings fell outside the expected range and were being flagged and set aside. The discovery was delayed. Say it: *“An automatic outlier filter can delete a Nobel-worthy discovery and leave no trace that it did.”*

The 999 example is the opposite pole and is the far more common case in student projects. Sentinel values (999, −1, 0, 9999, 1900-01-01) are everywhere in real administrative data.

Ask: “What is the difference between these two cases?” Steer them to: in one, the process that generated the number was the real world; in the other, the process was a data-entry convention.

Three types of outliers

Point (global)	Contextual	Collective
One value is extreme compared to the whole dataset.	The value is normal overall, but abnormal <i>in its context</i> .	No single value is odd, but a <i>group</i> of them together is.
<i>A \$45,000 monthly rent in a list of student apartments.</i>	<i>28°C is fine in July. In January in DC it is an outlier.</i>	<i>A heart rate of 65 is fine. Being flat at exactly 65 for 3 hours is not.</i>
Detected by: z-score, IQR, MAD	Detected by: group-wise rules, residuals from a model	Detected by: sequence / pattern methods

Why this matters today

Our Example 3 is a **contextual** outlier. Every univariate method you are about to learn will look straight at it and report “nothing unusual.”

Taxonomy (3 min).

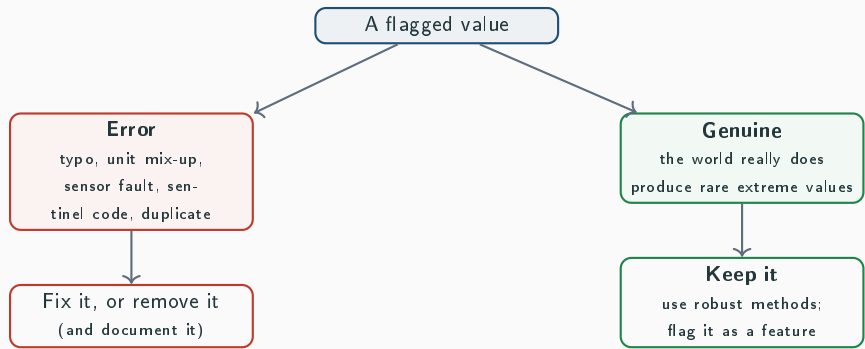
Students only ever learn point outliers in intro stats, so the other two columns are the new material. Do not let the three columns blur together — give each one its concrete example and move on.

Contextual: the temperature example works for any city. Ask: “What column would you need in order to notice this?” Answer: the month. The outlier only exists relative to a second variable. That is the whole idea.

Collective: the flat heart rate is a real thing — it usually means a sensor disconnected and is repeating its last value. Another good one: 500 identical transactions of \$9.99 in one minute; each is ordinary, the burst is fraud.

The closing alert block sets up Example 3. Tell them to remember this slide when you get there, because the punchline of Example 3 is that the tools from the next section find nothing.

The fork in the road: where did it come from?



The question statistics cannot answer for you

“Is this value *possible*?” is a domain question. A 47-minute half marathon is physically impossible for an amateur. A \$45,000 rent is possible — in the wrong dataset.

Decision framework (3 min). This diagram is the spine of the lecture. Draw it again on the board later from memory; repetition is what makes it stick.

Make the branch concrete with the current world record: the men’s half-marathon record is around 57 minutes, set by an elite professional. So 47 minutes is not merely unlikely for a campus race — it is faster than any human has ever run the distance. That is not a statistical judgment, it is a *domain* judgment, and no formula on the next slides could have told you.

Ask the class: “What are the three most likely explanations for the 47?” Harvest answers: (a) she cut the course, (b) bib mix-up / wrong person timed, (c) someone typed 47 instead of 147, (d) the clock recorded a 10K split. All four are *errors*, but they call for different fixes — and (c) is recoverable if you can find the original sheet.

Emphasize “document it.” A dropped row with no note is unreproducible research. Later in the course we will require a written justification for every removed observation.

Three Ways to Detect Them

$$z_i = \frac{x_i - \bar{x}}{s}$$

Rule of thumb: flag $|z_i| > 3$.

Reading it: z_i is how many standard deviations x_i sits from the mean. Under a normal distribution, only about 0.3% of values lie beyond ± 3 .

Assumes: roughly normal data, and a large enough sample.

```
z = (x - x.mean()) / x.std(ddof=1)
flagged = x[abs(z) > 3]
```

The hidden circularity

$\bar{x}$ and s are computed *from the data that contains the outlier*.

The outlier drags $\bar{x}$ toward itself and inflates s — and then we divide by that inflated s . The outlier helps to hide itself.

This is called **masking**, and Example 2 will show it doing real damage.

z-score (4 min).

Most students have seen this formula. Your job is not to re-teach it — it is to make them distrust it.

Make the circularity physical. Ask a student to imagine the number line with 14 points clustered near 100 and one at 47. The mean is a balance point, so it slides left toward the 47. Then the SD is measured *from that shifted mean*. Both ingredients of the z-score have been corrupted by the very point we are testing.

Note ddof=1. Pandas defaults to the sample SD ($n - 1$); NumPy’s .std() defaults to the population SD (n). Mixing them silently changes your answers. This bites students in homework every single semester — say it now and say it again in lab.

Preview: “On the next slide I am going to show you a sample size where this method is mathematically incapable of flagging anything at all.”

Method 1 has a hard ceiling

A fact most people never learn

For a sample of size n , the largest possible z-score is

$$|z|_{\max} = \frac{n - 1}{\sqrt{n}}$$

No matter how extreme the value is. Not approximately — *exactly*.

n	$ z _{\max}$	Can the “ $ z > 3$ ” rule ever fire?
10	2.85	Never
12	3.18	Barely — only near the theoretical limit
20	4.25	Yes
50	6.93	Yes

Consequence

With $n \leq 10$, “no $|z| > 3$ ” tells you **nothing about your data**. It is a fact about arithmetic. Yet this is exactly the sample size of a typical class project.

The ceiling (4 min). This is the slide students remember.

Let the table sit for a moment, then say: *“If you have ten data points and you run the three-sigma rule, you will find nothing. Not because your data is clean. Because it is impossible.”*

Sketch of why (do this only if the room is with you — it takes 90 seconds). Put one point at the extreme and the other $n - 1$ points together. The z-score is maximized when all remaining points are identical. Working through the algebra of $\sum z_i = 0$ and $\sum z_i^2 = n - 1$ gives the bound $(n - 1)/\sqrt{n}$. Do not belabor the derivation; the *existence* of the ceiling is the takeaway.

Practical advice to give them: for $n < 30$, do not use the z-score at all. Use the IQR rule or the modified z-score coming up next.

Flag that Example 2 uses $n = 12$ deliberately, so they will watch this ceiling cause an actual failure in about ten minutes.

Recipe

1. $Q_1 = 25\text{th percentile}$, $Q_3 = 75\text{th percentile}$
2. $\text{IQR} = Q_3 - Q_1$
3. Fences:
$$\text{lower} = Q_1 - 1.5 \times \text{IQR}$$
$$\text{upper} = Q_3 + 1.5 \times \text{IQR}$$
4. Flag anything outside the fences

Use $3.0 \times \text{IQR}$ for “far out” / extreme outliers.

```
q1, q3 = x.quantile(0.25), x.quantile(0.75)
iqr = q3 - q1
flagged = x[(x < q1 - 1.5*iqr) | (x > q3 + 1.5*iqr)]
```

Why it is better

Quartiles do not care how far away the outlier is. Move the 47 to 4.7 or to -1000 — Q_1 , Q_3 , and the fences *do not change at all*.

This is what a boxplot draws

The box is Q_1 to Q_3 ; the whiskers stop at the fences; the dots beyond are the flagged points. You have been running this method by eye for years.

IQR (4 min).

Do the “move the outlier” demonstration live. In the notebook, change the 47 to -5000 and re-run the fence calculation. The fences are identical. Then do the same with the z-score and watch every z-score change. Seeing it beats hearing it.

Where does 1.5 come from? Somebody always asks. Honest answer: Tukey chose it by judgment, not derivation. For normal data it flags about 0.7% of points, which he considered a reasonable rate of false alarms. It is a convention, not a law — which is why $3.0 \times \text{IQR}$ exists for a stricter screen.

Warn about quartile conventions. There are at least nine definitions of a quartile in use. NumPy’s linear interpolation and the textbook “ $(n + 1)/4$ position” rule give slightly different numbers. You will see this in Example 1: $Q_1 = 97$ by hand versus 97.5 in NumPy. Tell them the conclusion rarely changes, but their by-hand answer may not match their code, and that is expected — not a bug.

Replace the mean with the **median** and the SD with the **median absolute deviation**:

$$\text{MAD} = \text{median}(|x_i - \tilde{x}|)$$
$$M_i = \frac{0.6745 (x_i - \tilde{x})}{\text{MAD}}$$

Rule of thumb: flag $|M_i| > 3.5$.

The constant 0.6745 rescales MAD so that M_i is comparable to a z-score for normal data.

```
med = x.median()
mad = (x - med).abs().median()
m_score = 0.6745 * (x - med) / mad
flagged = x[m_score.abs() > 3.5]
```

Breakdown point	
The fraction of your data that can be garbage before the method itself breaks:	
Mean / SD	0%
IQR fences	25%
Median / MAD	50%

One bad value is enough to break the mean. Half your dataset must be corrupt to break the median.

Modified z-score (4 min).

Frame it as “the z-score, repaired”: same shape of formula, but both fragile ingredients swapped for robust ones.

The breakdown point table is the real content here. Define it plainly: how much of the data an adversary would have to corrupt before your statistic becomes arbitrary. For the mean the answer is one point — I can make the mean anything I want by changing a single value. For the median I would need to corrupt half the dataset.

Where 0.6745 comes from: for a normal distribution, $\text{MAD} \approx 0.6745\sigma$. Dividing by it puts M back on the familiar z-score scale, which is why the cutoff (3.5) looks like the z-score cutoff (3).

The one trap: if more than half your values are identical, $\text{MAD} = 0$ and you divide by zero. This is common with counts and with zero-inflated data (e.g. “number of complaints” where most rows are 0). In the notebook we guard for it. Mention it now so nobody loses an hour to it later.

Choosing a method

	z-score	IQR fences	Modified z (MAD)
Built from	mean, SD	quartiles	median, MAD
Breakdown point	0%	25%	50%
Assumes normality	yes	no	loosely
Small n (< 20)	fails	fine	fine
Skewed data	poor	fair	good
Multiple outliers	masking	robust	robust
Interpretation	“SDs from mean”	“outside the box”	“robust SDs”

Practical default

Start with a **boxplot**. Then run **IQR** and **modified z-score** together. If they disagree, that disagreement is information — go look at the actual rows.

Comparison (2 min). Do not read the table aloud row by row — they can read. Point at the two red cells in the z-score column and say: “these two rows are why professionals rarely use the z-score for outlier screening.”

Tell them to photograph this slide. It is the reference table for the homework.

On the “practical default” box: emphasize *look at the rows*. Running three detectors and reporting counts is not data wrangling. Opening the flagged rows and asking “what happened here?” is. Methods disagreeing usually means the value is borderline, and borderline cases are exactly the ones that require judgment rather than a threshold.

Transition line: “Let’s put all three to work on three real problems.”

Example 1 — The Bison Half-Marathon

Example 1 — The data

Setup. Fifteen runners in one age group. Times in minutes. The race director asks you to check the results before they are posted.

92, 95, 97, 98, 101, 103, 104, 106, 108, 110, 112, 115, 118, 121, 47

Sorted ($n = 15$):

47	92	95	97	98	101	103	104	106	108	110	112	115	118	121
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Positions: Q_1 at $\frac{n+1}{4} = 4$, median at $\frac{n+1}{2} = 8$, Q_3 at $\frac{3(n+1)}{4} = 12$.

Example 1 setup (2 min).

Have the class sort the numbers with you — it takes twenty seconds and it makes the quartile positions concrete rather than symbolic.

Point out that the position formula $(n + 1)/4$ lands on a whole number here ($n = 15$) precisely because this example was designed that way. With $n = 12$ you would land between two observations and have to interpolate. Say that you chose $n = 15$ so the arithmetic stays clean while they learn the method.

Before computing anything, ask: “Just by eye, which value are we going to flag?” Everyone says 47. Good — then the point of the next slide is not *finding* it but *justifying* it with a number you could defend to the race director, or in court.

Set the frame: “You are not doing statistics for its own sake. You are about to tell someone their result is being disqualified. You need a defensible rule.”

Example 1 — Working the IQR rule by hand

Step 1 — Quartiles

$Q_1 = 97 \qquad Q_3 = 112$

Step 2 — IQR

$IQR = 112 - 97 = 15$

Step 3 — Fences

$lower = 97 - 1.5(15) = 74.5$

$upper = 112 + 1.5(15) = 134.5$

Step 4 — Compare

$47 < 74.5 \Rightarrow \text{FLAGGED}$

And the verdict?

The fastest half marathon ever run is about 57 minutes. A 47-minute finish is not rare — it is impossible. This is an error: left branch, investigate and fix.

Check against Python

NumPy interpolates quartiles differently:

	By hand	NumPy
Q_1	97	97.5
Q_3	112	111.0
IQR	15	13.5
Lower	74.5	77.25

Different numbers, same conclusion. Both flag 47 and nothing else.

Example 1 worked (5 min). Do every arithmetic step on the board.

Students who only ever see `df.describe()` never internalize what a fence is. Make them watch $1.5 \times 15 = 22.5$ get subtracted.

The By hand vs NumPy box matters more than it looks. Students panic when their homework answer differs from their code in the second decimal. Normalize it now: different quartile conventions, same conclusion. If the conclusion *did* flip, that tells you the point is borderline and deserves a human look — which is itself useful information.

Close the loop on the decision framework. Walk back to the fork diagram: the statistics flagged it, the domain knowledge (world record ≈ 57 min) classified it as impossible, and *that* is what licenses removal. Ask: “What would you do next if you were the race director?” Best answer: check the timing mat splits and the bib assignment before you do anything to the data — fixing beats deleting.

If time permits: ask what would change if this were a *wheelchair* division. 47 minutes becomes entirely plausible. Same number, different context, opposite verdict.

Example 2 — The \$1 Apartment

Example 2 — The data

Setup. Twelve monthly rents (\$) scraped from a listings site for apartments near campus. You are computing “average rent near campus” for a student housing report.

1800, 1950, 2000, 2100, 2150, 2200, 2250, 2300, 2400, 2500, 1, 45000

Two suspicious values

- **\$1** — a placeholder a landlord typed to make the listing appear in “lowest price” sorts
- **\$45,000** — an annual rent entered in a monthly field

The naive answer

$$\bar{x} = \$5,554.25$$

Ten of the twelve apartments rent for \$2,500 or less. The “average” is more than double every one of them.

Question: will the $|z| > 3$ rule catch them?

Example 2 setup (2 min).

Both error types here are *real* scraping problems, not invented ones. The \$1 placeholder is a well-known listing-site tactic to win price-sorted searches; the annual-in-a-monthly-field mistake is one of the most common unit errors in housing data.

Let the \$5,554 average land. Ask: “If you published this number in the student newspaper, what happens?” Someone will say students conclude they cannot afford to live near campus. That is the cost of not screening — a wrong number that changes someone’s decision.

Poll the room before the reveal: “Hands up if you think the three-sigma rule catches the \$45,000. Hands up for the \$1.”

Most hands go up for both. Write the vote on the board so you can point at it when the answer arrives. $n = 12$ — remember the ceiling slide.

Example 2 — The z-score fails

$\bar{x} = 5554.25,$ $s = 12439.24$

Rent	z
1800	−0.302
2000	−0.286
2200	−0.270
2500	−0.246
1	−0.446
45000	+3.171

Two failures at once

1. The \$1 is invisible. Its z-score is −0.446 — more ordinary-looking than most of the real rents. *Masking*.

2. The \$45,000 barely squeaks through. $z = 3.171$, and for $n = 12$ the *maximum possible* z-score is

$$\frac{n-1}{\sqrt{n}} = \frac{11}{\sqrt{12}} = 3.175$$

So every z-score the rule can ever flag lives in the window (3.000, 3.175) — a target 0.175 wide. This error landed at 99.9% of the ceiling.

Read that again

A value 45,000 times larger than another value in the same column was almost too small to flag.

The failure (5 min). This is the emotional peak of the lecture — slow down and let it hurt.

Take the two failures one at a time.

The \$1: its z-score is −0.446, while a perfectly normal \$2,500 rent scores −0.246. The typo looks *more* typical than the real data. Ask why: because the \$45,000 dragged the mean up to \$5,554 and blew the SD up to \$12,439. Against a yardstick that big, the distance from \$5,554 down to \$1 is nothing. The big outlier is hiding the small one. That is masking, and it is why we do not screen with the mean.

The \$45,000: point back at the ceiling slide and the vote on the board. 3.171 out of a possible 3.175 — as extreme as the arithmetic permits. The window the rule can fire in runs from 3.000 up to 3.175 and no further, so it is 0.175 wide. Say it plainly: “Set your cutoff at 3.2 — a value plenty of textbooks use — and this rule can never flag anything at all in a sample of twelve, no matter how broken the data is.”

Do not let them conclude “use a lower cutoff.” Ask what happens at $|z| > 2$: you start flagging \$1,800 and \$2,500 too. The problem is not the threshold, it is the ingredients. Next slide fixes the ingredients.

Example 2 — The modified z-score succeeds

Robust ingredients

$\tilde{x} = 2175$ $\text{MAD} = 200$

$$M_i = \frac{0.6745 (x_i - 2175)}{200}$$

Rent	M	
1800	−1.26	
2000	−0.59	
2200	+0.08	
2500	+1.10	
1	−7.33	FLAG
45000	+144.43	FLAG

Both caught, decisively

Not marginally. The \$1 scores −7.33 against a cutoff of 3.5, and the \$45,000 scores 144.

The IQR rule also works

$Q_1 = 1987.5, Q_3 = 2325, \text{IQR} = 337.5$

Fences: (1481.25, 2831.25)

Flags \$1 and \$45,000, keeps all ten real rents.

Corrected mean of the ten real rents: \$2,165

The fix (4 min).

Compute MAD with them, slowly, because the “median of the absolute deviations from the median” phrase is genuinely confusing the first time:

- 1. Median of the rents: with $n = 12$, average the 6th and 7th sorted values, $(2150 + 2200)/2 = 2175$.
- 2. Distance of every value from 2175: 2174, 375, 225, 175, 75, 25, 25, 75, 125, 225, 325, 42825.
- 3. Median of *those* distances: $(175 + 225)/2 = 200$. That is MAD.

Point out what just happened: the \$45,000 contributed the distance 42,825 to that list, and taking a median *threw it away*. The outlier had no vote in setting the yardstick. Contrast with the SD, where it set the yardstick almost single-handedly.

Land the comparison: same data, same question, one method says “−0.446, looks fine” and the other says “−7.33, obviously broken.”

The corrected \$2,165 is the number that should have gone in the housing report. Ask: “What do you write in the methods section?” Answer: that you excluded two listings, which two, and why — a \$1 placeholder and an apparent annual figure.

Three sentences. That is reproducible research.

Example 3 — The Ice Cream Truck

Example 3 — The data

Setup. An ice cream truck parks outside the library. For 20 days the owner logs the temperature and units sold. She wants a model: how much should she stock tomorrow?

Day	Temp (C)	Sales	Day	Temp (C)	Sales
D01	15.0	210	D11	25.0	399
D02	16.0	233	D12	26.0	414
D03	17.0	241	D13	27.0	428
D04	18.0	248	D14	82.4	427
D05	19.0	274	D15	29.0	461
D06	20.0	282	D16	30.0	493
D07	21.0	319	D17	31.0	474
D08	22.0	360	D18	32.0	508
D09	23.0	345	D19	20.0	450
D10	75.2	361	D20	24.0	349

Two different problems are hiding in this table. One is an inconsistency. One is a contextual outlier.

Example 3 setup (3 min).

Give them 60 seconds to find the problems themselves before you say anything. Put the timer up. This is the first example where the answer is not obvious from a glance at one column.

Expect them to find D10 and D14 quickly — 75.2°C and 82.4°C are absurd temperatures. Ask: “What are those numbers really?” Someone will get it: Fahrenheit. $75.2^{\circ}\text{F} = 24^{\circ}\text{C}$ and $82.4^{\circ}\text{F} = 28^{\circ}\text{C}$. Two rows were entered from a different thermometer. This is a **unit inconsistency**, our bridge to the next section.

Almost nobody finds D19 unaided. When someone does, ask them how they saw it — the answer is always “I compared it to D06,” which had the same temperature and sold 282. That comparison *is* contextual outlier detection, done by eye.

Do not resolve D19 yet. Let it sit while you show that the standard tools miss it.

A validity rule catches it instantly

Column	Plausible range
temp_c	−20 to 45 °C
sales	0 to 2000 units

Violations: **D10 (75.2)** and **D14 (82.4)**

The fix — convert, do not delete:

$$C = \frac{(F - 32) \times 5}{9}$$

75.2 → 24.0 82.4 → 28.0

Why this is not an “outlier” problem

Nothing about the ice cream business is unusual on those days. The *measurement process* was inconsistent. Two rows used a different unit. Deleting them would throw away two perfectly good observations.

\$327 million

The Mars Climate Orbiter was lost in 1999 because one team used pound-force seconds and another used newton-seconds. Unit inconsistency is not a small-data problem.

Inconsistency (3 min).

The central lesson: a validity rule beats a statistical detector when you know the domain. You did not need quartiles or z-scores here. You needed to know that water boils at 100°C and that Washington DC has never hit 75°C. One line of code, total certainty.

Tell them to write range rules for every numeric column *before* they run any detector. Ages 0–120. Percentages 0–100. Counts ≥ 0. Dates not in the future. It takes five minutes and catches the errors no statistic will.

Emphasize convert, do not delete. The reflex to drop bad rows is strong. Here dropping costs you 10% of a 20-row dataset for no reason — the observations were fine, the encoding was not.

The Mars Climate Orbiter is worth thirty seconds. NASA lost a \$327M spacecraft in 1999 because Lockheed Martin’s software produced impulse in pound-force seconds and NASA’s navigation software expected newton-seconds. The orbiter entered the Martian atmosphere too low and was destroyed. Ask: “Would a range check have caught that?” Probably yes — and nobody wrote one.

Example 3 — Problem 2: the invisible outlier

D19: 20°C, 450 units sold. Run every univariate method on the sales column:

Method	Result for D19	Flagged?
Range check	$0 \leq 450 \leq 2000$	No
z-score	$z = 0.93$ (cut off 3)	No
IQR fences	fences = (49.8, 663.8); 450 is inside	No
Boxplot	well inside the whiskers	No
Max sales in the data	508 units (D18) — and $450 < 508$	

Every tool from this lecture says the value is fine

And every tool is **wrong**. 450 units is perfectly ordinary — for a 29°C day. On a 20°C day, the truck sells about 282.

The outlier only exists in relation to the temperature column.

The invisible outlier (4 min). This slide is the reason Example 3 is in the lecture.

Read the table as a series of defeats. Range check: fine. z-score: 0.93, not remotely close. IQR: sits comfortably inside fences that are 600 units wide. Boxplot: nowhere near a whisker.

Say the uncomfortable thing: “If you had run the standard outlier checklist on this dataset — the checklist I taught you twenty minutes ago — you would have reported a clean column and moved on.”

Make the comparison explicit. D06 is also 20°C and sold 282 units. D19 sold 450 at the same temperature. That is 60% more ice cream on an identical day. The number is only strange *next to* the temperature.

Ask: “So what do we do?” Guide them: if a single column cannot see it, stop looking at single columns. Model the relationship, then look at what the model fails to explain. That is the residual method on the next slide — and it is the conceptual bridge from data cleaning to modeling.

Example 3 — Finding it with residuals

Step 1. Fix the units first (D10, D14).

Step 2. Fit sales against temperature:

$$\widehat{\text{sales}} = -60.5 + 17.8 \times \text{temp_c}$$

Step 3. Residual = actual – predicted.

Step 4. Standardize the residuals and flag $|r| > 3$.

Day	Pred.	Resid.	r
D06 (20°)	296.3	−14.3	−0.39
D16 (30°)	474.8	+18.2	+0.50
D19 (20°)	296.3	+153.7	+4.21

Now it is obvious

Every other day sits within half a standard deviation of the line. D19 sits 4.2 standard deviations above it — it sold 154 more units than any 20°C day should.

So what happened on D19?

The owner checks her calendar: it was Homecoming. Ten thousand extra people walked past the truck. The value is real. It is not an error.

Residual method (4 min).

Step 1 is not optional and students will skip it. If you fit the line before converting D10 and D14, those two rows sit at 75° and 82° on the x-axis with ordinary sales, and they drag the fitted line flat. The fit becomes garbage and D19 may no longer stand out. Order of operations: fix inconsistencies first, then detect outliers. Write that on the board.

Keep the regression light — most of them have seen a line of best fit. The idea is all that matters: the line encodes “what we expect at this temperature,” and the residual is “how much reality disagreed.” Outlier detection on residuals is just the univariate methods applied to the disagreement instead of the raw value.

Note the contrast: $z = 0.93$ on raw sales, $r = 4.21$ on the residual. Same observation, same dataset. The difference is entirely in what you compared it to.

The Homecoming reveal should be a beat of its own. Pause before you say it. The detective work is the payoff.

Example 3 — The verdict

The wrong move

Delete D19. The model gets a nicer R^2 . It also now predicts 296 units for next Homecoming — and the truck runs out of ice cream by noon.

The right move

Keep it, and explain it. Add an event_day column. Now D19 is not an unexplained outlier — it is the single most valuable row in the dataset, because it is the only evidence you have about what events do to demand.

The general principle

An outlier you cannot explain is a data quality problem.
An outlier you *can* explain is a **missing variable**.

Ask “what is different about this row?” before you ask “how do I remove this row?”

Verdict (3 min). This is the moral of the entire lecture — deliver it deliberately.

Make the cost of deletion concrete. If you drop D19 and model only ordinary days, your forecast for the next big campus event is 296 units. The truth was 450. The owner under-stocks by a third and loses a day of peak revenue. The “cleaner” dataset produced the more expensive decision.

Then the reframe: the outlier was never really about ice cream sales. It was the dataset telling you that a variable you failed to record — whether a major event was happening — matters enormously. Outliers are frequently *signals of omitted variables*.

Read the principle box aloud. Then repeat the operational version: “*Ask what is different about this row before you ask how to remove it.*”

Good closing question: “Where else does this pattern show up?” Harvest answers — fraud detection (the outlier is the entire point), rare disease diagnosis, equipment failure prediction, record-breaking athletic performance. In every one of those fields, deleting outliers means deleting the thing you were hired to find.

Handling What You Find

Five things you can do (and when)

Action	Use when	Cost / risk
Fix	You can recover the true value (unit conversion, re-read the source)	Best option. Requires access to the source.
Remove	Confirmed error, unrecoverable, few rows	Shrinks <i>n</i> ; biases results if the errors are not random
Cap (winsorize)	Genuine extremes you want to limit, not erase	Distorts the tail; never do it silently
Transform	Right-skewed positive data (income, counts)	log compresses extremes; changes interpretation of every coefficient
Keep + flag	Genuine and meaningful	Usually correct; add an indicator column

Non-negotiable

Whatever you choose: **record how many rows were affected and why**, and report your headline result *with and without* the change.

Handling (4 min).

Walk the table top to bottom, and stress that it is *ordered by preference*. Fixing beats removing. Removing is the option students reach for first and should reach for last.

Winsorizing: define it — replace everything above the 95th percentile with the 95th percentile value, and likewise at the bottom. Useful when extremes are real but you do not want them dominating a mean. Dangerous because your data now contains values that were never observed. If you report a maximum after winsorizing, you are reporting a number that did not happen.

Log transform: the standard move for income, house prices, city populations, word frequencies. Warn them that coefficients then describe multiplicative effects, and that log(0) is undefined — use `np.log1p` for count data with zeros.

The sensitivity analysis in the alert block is the professional standard. “Mean rent is \$2,165 excluding two listings flagged as data errors; including them it is \$5,554.” That sentence takes ten seconds to write and makes your work honest and checkable. Tell them it is required on every project in this course.

Inconsistencies and Data Quality

Inconsistencies: same meaning, different bytes

Type	What it looks like	Fix
Units	75.2 and 24.0 in one temperature column	Range rule, then convert
Category variants	"NY", "N.Y.", "new york", "New York "	.str.strip().str.lower(), then a mapping dictionary
Date formats	03/04/2026 — March 4th or April 3rd?	Parse with an explicit format; store ISO YYYY-MM-DD
Duplicates	Same order ID twice; same person, two spellings	.duplicated() on a key; fuzzy match on names
Cross-field logic	qty * price != total; ship_date < order_date	Write the rule as an assertion and test every row
Sentinels	999, -1, "N/A", 1900-01-01	Convert to real missing values (NaN)

The pattern

Outliers hide in *numbers*. Inconsistencies hide in *encodings* — and a statistical detector will never find them.

Inconsistencies (5 min). This is the most immediately useful slide for their projects.

Go quickly but give each row one war story:

- **Categories:** a survey with a free-text state field will produce dozens of spellings of one state. Count unique values before you group by anything — `df['state'].value_counts()` is the fastest data quality check there is.
- **Dates:** 03/04/2026 is March 4th in the US and April 3rd in the UK. If a dataset mixes both conventions, roughly a third of the dates are silently wrong and *nothing* will flag them. Always pass an explicit `format=` to `pd.to_datetime`.
- **Cross-field:** the single highest-yield check in business data. `qty * unit_price` should equal `total`. When it does not, you have found a discount column nobody told you about, or a bug.
- **Sentinels:** a 999 that survives into a mean will quietly poison it. Worse, it looks like a legitimate number to every detector.

Land the closing box: no amount of quartile arithmetic will notice that "NY" and "New York" are the same place. Different class of problem, different class of tool.

Assessing data quality: six dimensions

1. **Completeness** — how much is missing?

```
df.isna().mean()
```

2. **Validity** — does it obey the rules?

```
age in [0, 120]; % in [0, 100]
```

3. **Consistency** — do fields agree?

```
qty*price == total
```

4. **Uniqueness** — any duplicates?

```
df.duplicated(subset='id')
```

5. **Accuracy** — does it match reality?

```
needs an external source
```

6. **Timeliness** — is it current enough?

```
how stale is the newest row?
```

Make it a habit

Write a `quality_report(df)` function once.

Run it on *every* dataset before you analyze anything.

```
def quality_report(df):  
    return pd.DataFrame({  
        "dtype": df.dtypes,  
        "missing_%": df.isna().mean()*100,  
        "unique": df.nunique(),  
        "min": df.min(numeric_only=True),  
        "max": df.max(numeric_only=True),  
    })
```

The full version is in the notebook.

Accuracy is the hard one: the other five you can check from inside the file. Accuracy requires the outside world.

Data quality (4 min).

Walk the six dimensions, but spend your time on the distinction at the bottom. Five of the six are **internal** checks — everything you need is in the file. **Accuracy** is different: a dataset can be complete, valid, consistent, unique, and current, and still be entirely wrong, because every value was measured with a broken instrument. Checking accuracy requires an external reference, which is why it is usually the dimension that gets skipped.

Push the habit hard. Tell them to put `quality_report` in a utilities module this week and run it as the first cell of every notebook for the rest of the semester. The report takes three seconds and routinely catches problems that would otherwise surface a week later inside a model.

Tie back to the examples: the \$1 rent is a *validity* failure, the Fahrenheit temperatures are a *consistency* failure, the Homecoming day is none of these — it is real data. Three different dimensions, three different responses. That mapping is exactly what you want them to internalize.

Practice Problems

Practice Problem 1 — The impossible exam

Scenario. You are the TA for a course scored out of 100. Here are 14 scores from one section:

72, 68, 91, 85, 77, 80, 88, 74, 95, 83, 105, -5, 79, 86

1. Write a **validity rule** for this column. Which values violate it?
2. Compute Q_1 , Q_3 , IQR, and the Tukey fences **by hand**. Does the IQR rule flag the same values your validity rule did?
3. The 105 turns out to be a legitimate score of 95 plus 10 bonus points. The -5 was a placeholder for “did not sit the exam.” **Handle each one** — and say which of the five actions from the handling table you chose and why.
4. Recompute the class mean before and after. Write the one sentence you would put in your report.

Practice 1 (5 min, in pairs).

The teaching point is that parts 1 and 2 disagree, and that is deliberate. The validity rule ($0 \leq \text{score} \leq 100$) catches both bad values instantly. The IQR rule catches -5 but **does not flag 105** — run it and see. With scores spread from 68 to 95, the upper fence lands above 105. Students find this genuinely surprising.

Moral: *a domain rule can be stronger than a statistical rule*. You know the scale maxes at 100. No amount of quartile arithmetic knows that.

Part 3 is the real exercise. The two values look similar and require opposite treatment:

- 105: **keep** it. It is not an error at all — the scale simply allows bonus points. The right fix is to correct your *rule*, not your data.
- -5: **convert to NaN**. It is a sentinel. It is not a score of negative five, and averaging it in is nonsense.

Part 4: mean with everything ≈ 76.6 ; with -5 as missing ≈ 82.5 . The one-sentence report is the deliverable — something like “one student did not sit the exam and is excluded from the average; one score above 100 reflects bonus credit and is retained.”

Circulate while they work. The common wrong answer is deleting both.

Practice Problem 2 — The clinic intake sheet

Scenario. Ten rows from a clinic’s patient table.

id	name	age	visit_date	bp_systolic
P01	Ana Reyes	34	2026-03-11	118
P02	Ben Osei	51	2026-03-11	132
P03	Cara Lin	999	2026-03-12	121
P04	Dan Meyer	45	03/12/2026	0
P05	Ana Reyes	34	2026-03-11	118
P06	Eve Duarte	29	2026-03-13	240
P07	Femi Ade	62	2026-13-14	128
P08	Gus Park	-3	2026-03-14	119
P09	Hana Cruz	47	2027-03-15	125
P10	Ivan Petrov	38	2026-03-15	134

Find every problem. There are at least **seven**. For each one: name the **quality dimension** it violates, and state your fix.

Hint: two of the problems are invisible if you only look one column at a time.

Practice 2 (7 min, in pairs). This is the richest problem — give it the most time.

The seven (there are eight if they are sharp):

1. P03 age = 999 — sentinel. *Validity.* → NaN.
2. P08 age = −3 — impossible. *Validity.* → NaN or source check.
3. P04 bp_systolic = 0 — a living patient has no blood pressure of zero. *Validity.* → NaN. Note it would *not* be caught by an “is it a number” check.
4. P06 bp_systolic = 240 — *plausible* (hypertensive crisis). Statistically an outlier, medically real. **Keep and flag.** This is the Homecoming row of this dataset.
5. P07 visit_date = 2026-**13**-14 — month 13. *Validity.*
6. P09 visit_date in **2027** — a future date. *Timeliness/validity.* Probably a typo for 2026.
7. P04 date written 03/12/2026 while every other row is ISO. *Consistency.*
8. P05 duplicates P01 (note the double space in "Ana Reyes"). *Uniqueness* — and it survives an exact-match dedupe, which is why whitespace normalization must come first.

The hint points to #6 and #8. The 2027 date is only wrong relative to the other rows, and the duplicate only appears once you compare rows.

Best discussion question: “Which of these would a describe() call have shown you?” Answer: the numeric ones, maybe. None of the date or duplicate problems. That is why the quality report has to check more than summary statistics.

Practice Problem 3 — The online store

Scenario. An e-commerce orders table. Every row *looks* fine column by column.

order	city	qty	unit_price	total	ship_date
A100	Washington	2	19.99	39.98	2026-04-02
A101	washington	1	45.00	45.00	2026-04-02
A102	Silver Spring	3	12.50	37.50	2026-04-03
A103	WASHINGTON	4	19.99	79.96	2026-04-01
A104	Silver Spring	2	30.00	66.00	2026-04-04
A105	Washington DC	1	99.99	99.99	2026-04-05
A103	WASHINGTON	4	19.99	79.96	2026-04-01

Order date for every row is 2026-04-02.

1. Group by `city` and count orders. Why is the answer wrong? Write the cleaning code.
2. Write a **cross-field assertion** linking `qty`, `unit_price`, and `total`. Which row fails?
3. One row ships *before* it was ordered. Find it.
4. One order ID appears twice. Is it a duplicate row, or a real second order? **How would you decide?**

Practice 3 (6 min). The theme: nothing here is an outlier, and the dataset is still broken.

Part 1 — categories. Four spellings of one city: Washington, washington, WASHINGTON, Washington DC. A naive groupby reports four cities where there is one. Fix: `.str.strip().str.lower()` handles three of them; the fourth needs an explicit mapping dictionary, because no automatic rule knows that “washington dc” and “washington” are the same market. **Make this point — normalization gets you most of the way, domain knowledge finishes the job.**

Part 2 — cross-field. A104: $2 \times 30.00 = 60.00$, but the total says 66.00. Ask what could explain the extra \$6. Sales tax? Shipping? A pricing bug? The assertion does not tell you *which* — it tells you to go ask somebody. That is the correct outcome.

Part 3 — temporal logic. A103 ships 2026-04-01, ordered 2026-04-02. Shipped a day before it was ordered. Impossible, and invisible to any check that looks at one column.

Part 4 is the discussion, and there is no single right answer. A103 appears twice, identical in every field. Arguments for a true duplicate: an order ID should be unique by definition, and every field matches exactly. Arguments for a real second order: a customer could genuinely order the same thing twice. **The resolution is procedural, not statistical** — check whether the ID is a primary key in the source system, or look for two payment records. Push them to say “I would go ask” rather than guessing. Knowing when the data cannot answer the question is a skill.

Wrap-Up

What to carry out of this room

1. **Investigate, do not delete.** A flagged value is a question, not a verdict.
2. **The z-score is the weakest tool you have.** Zero breakdown point, and for $n \leq 10$ it is mathematically incapable of flagging anything.
3. **Prefer robust methods:** IQR fences and the modified z-score.
4. **Fix inconsistencies before detecting outliers.** Units first, statistics second.
5. **Some outliers are invisible one column at a time.** If nothing looks wrong, compare columns.
6. **An outlier you can explain is a missing variable.**
7. **Document every change** and report results with and without it.

Coming up

Notebook: `outliers_data_wrangling.ipynb` — all three examples, plus the practice problems with worked solutions.

Wrap-up (3 min).

Do not read all seven. Pick three and say them with conviction — #1, #4, and #6 are the ones that change how students actually behave.

Call back to the three stories by name, since that is how they will remember the content: the marathon taught us that domain knowledge decides; the \$1 apartment taught us that the mean and SD can hide the very thing they are supposed to reveal; the ice cream truck taught us that some outliers live between columns, not inside them.

Assign: notebook first, then Practice Problems 1–3 written up. Remind them the deliverable for each problem includes the one-sentence justification, not just the code — that sentence is the part that is being graded hardest.

Preview next session and connect it: missing data. Note that today created missing values (every sentinel converted to NaN), so the natural next question is what to do with them — and that imputation has exactly the same “document your choices” discipline.

Questions?

Open the notebook and let's break some data.

Close.

Leave this up while students open the notebook. Move to the front of the room rather than staying behind the laptop.

If **nobody asks anything**, prime the pump with one of these:

- “What is the largest dataset any of you has cleaned? What broke?”
- “Where would deleting an outlier be genuinely dangerous?” (medicine, fraud, structural engineering, credit risk)
- “Has anyone ever had a result change after cleaning? By how much?”

Questions you should expect:

- “*Which method should I just always use?*” — IQR and modified z-score together, then look at the rows. There is no always.
- “*What if a method flags 20% of my data?*” — then the data is not unimodal and the method is wrong for it. Check for mixed populations or heavy skew; consider a transform or group-wise analysis.
- “*Can I automate all of this?*” — you can automate detection; you cannot automate the verdict. Automate the flagging, keep the human in the decision.